

Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Sáu, 23 tháng 8 2024, 3:17 PM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 3:19 PM
Thời gian thực hiện	2 phút 42 giây
Điểm	12,00/12,00
Điểm	10,00 trên 10,00 (100%)



Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

Implement function

```
void printArray(int n){}
```

to print 0, 1, 2, ..., n (n is positive integer and has no space at the end).

Please note that you can't using key work for, while, goto (even in variable names, comment).

For this exercise, we have #include <iostream> and using namespace std;

For example:

Test	Result
printArray(5);	0, 1, 2, 3, 4, 5
printArray(10);	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Answer: (penalty regime: 0, 0, 0, 5, 10, 15, ... %)

[Reset answer](#)

```
1 |
2 | void printArray(int n)
3 | {     static int i =0;
4 |     if(i==n) {
5 |         cout<<i;
6 |         return;}
7 |
8 |
9 |     cout<<i<<" ";
10 |    i++;
11 |    printArray(n);
12 |
13 | }
```



	Test	Expected	Got	
✓	printArray(5);	0, 1, 2, 3, 4, 5	0, 1, 2, 3, 4, 5	✓
✓	printArray(10);	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 2

Đúng

Đạt điểm 1,00 trên 1,00

Given a positive number, print following a pattern without using any loop.

Input: n = 16

Output: 16 11 6 1 -4 1 6 11 16 (has no space at the end)

Input: n = 10

Output: 10 5 0 5 10 (has no space at the end)

We basically first reduce 5 one by one until we reach a negative or 0. After we reach 0 or negative, we one add 5 until we reach n.

Note: Please note that you can't using key work for, while, goto (even in variable names, comment).

You can implement other recursive functions if needed.

For this exercise, we have #include <iostream> and using namespace std;

For example:

Test	Result
printPattern(14);	14 9 4 -1 4 9 14

Answer: (penalty regime: 0 %)

Reset answer

```

1 void printPatternSP(int n, int index, bool isdegree){
2     cout << n;
3     if (n == index && !isdegree){
4         return;
5     }
6
7     cout << " ";
8
9     if (isdegree){
10        if (n - 5 > 0){
11            return printPatternSP(n - 5, index, true);
12        }
13        else{
14            return printPatternSP(n - 5, index, false);
15        }
16    }
17    else{
18        return printPatternSP(n + 5, index, false);
19    }
20 }
21
22
23 void printPattern(int n){
24     printPatternSP(n, n, true);
25 }
```

	Test	Expected	Got	
✓	printPattern(14);	14 9 4 -1 4 9 14	14 9 4 -1 4 9 14	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

//



Câu hỏi 3

Đúng

Đạt điểm 1,00 trên 1,00

Implement function

```
int findMax(int* arr, int length){}
```

to find the largest element using recursion (with length is the number of elements in integer array arr).

Please note that you can't using key work for, while, goto (even in variable names, comment).

For this exercise, we have #include <iostream> and using namespace std;

For example:

Test	Result
<pre>int arr[] = {10, 5, 7, 9, 15, 6, 11, 8, 12, 2}; cout << findMax(arr, 10);</pre>	15

Answer: (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
1 int findMax(int* arr, int length) {  
2     if (length == 1){  
3         return arr[0];  
4     }  
5  
6     int findMaxofArray = findMax (arr + 1, length - 1);  
7  
8     if (arr[0] > findMaxofArray){  
9         return arr[0];  
10    }  
11    else {  
12        return findMaxofArray;  
13    }  
14 }  
15 }
```



	Test	Expected	Got	
✓	<pre>int arr[] = {10, 5, 7, 9, 15, 6, 11, 8, 12, 2}; cout << findMax(arr, 10);</pre>	15	15	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 4

Đúng

Đạt điểm 1,00 trên 1,00

Implement function

```
bool isPalindrome(string str){}
```

to check if the given non empty string is palindrome, else not palindrome using recursion.

In test case, for extra point, we will have some palindrome sentences (All remaining test cases are words).

Please note that you can't using key work for, while, goto (even in variable names, comment).

For this exercise, we have `#include <iostream>`, `#include <string>` and using namespace std;

For example:

Test	Result
cout << isPalindrome("mom");	1
cout << isPalindrome("do geese see god");	1

Answer: (penalty regime: 0 %)

Reset answer

```

1 // Hàm đệ quy để loại bỏ khoảng trắng và chuyển thành chữ thường
2 #include<string>
3
4
5 string preprocessString(const string& str, unsigned int index = 0) {
6     if (index == str.size()) { // neu chay den phan tu o ngoai thi bang rong
7         return "";
8     }
9     char c = str[index];
10    if (!isspace(c)) {
11        // return string(1, tolower(c)) + preprocessString(str, index + 1);
12        //string strtmp(1,tolower(c));
13        string strtmp = "";
14        strtmp+= tolower(c);
15        strtmp += preprocessString(str, index + 1);
16        return strtmp;
17    }
18    return preprocessString(str, index + 1);
19 }
20
21 // Hàm đệ quy kiểm tra palindrome
22 bool isPalindromeHelper(const string& str, unsigned int left, unsigned int right) {
23     // Trường hợp cơ bản
24     if (left >= right) {
25         return true;
26     }
27     // Kiểm tra ký tự đầu và cuối
28     if (str[left] != str[right]) {
29         return false;
30     }
31     // Gọi đệ quy kiểm tra phần còn lại của chuỗi

```



```
32     return isPalindromeHelper(str, left + 1, right - 1);
33 }
34
35 // Hàm chính để kiểm tra palindrome
36 bool isPalindrome(string str) {
37     string processedStr = preprocessString(str);
38     return isPalindromeHelper(processedStr, 0, processedStr.length() - 1);
39 }
```

	Test	Expected	Got	
✓	cout << isPalindrome("mom");	1	1	✓
✓	cout << isPalindrome("do geese see god");	1	1	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

//



Câu hỏi 5

Đúng

Đạt điểm 1,00 trên 1,00

Give two positive integers a and b, implement function

```
int findGCD(int a, int b){}
```

to find **GCD** (Greatest Common Divisor) of a and b using recursion.

Please note that you can't using key work for, while, goto (even in variable names, comment).

For this exercise, we have #include <iostream> and using namespace std;

For example:

Test	Result
cout << findGCD(124,32);	4

Answer: (penalty regime: 0 %)

Reset answer

```
1 int findGCD(int a, int b) {  
2     // Trường hợp cơ sở: nếu b == 0, GCD là a  
3     if (b == 0)  
4         return a;  
5  
6     // Đệ quy: GCD(a, b) = GCD(b, a % b)  
7     return findGCD(b, a % b);  
8 }
```



	Test	Expected	Got	
✓	cout << findGCD(124,32);	4	4	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 6

Đúng

Đạt điểm 1,00 trên 1,00

String s contains lowercase letters, digits, "(" and ")", satisfying the following rules:

- Two digits cannot be adjacent.
- Two "(" cannot be adjacent.
- One "(" and one ")" cannot be adjacent.
- After any digit, there must be "(".
- The quantities of "(" and ")" are equal.

Change string s until new string t created, t contains only lowercase letters. These are changing rules:

- Sub-strings with form " $n(p)$ ", can change to " $pp...p$ " (n times p), where n is a digit and p is a string.
- If p still contains "(", ")" or digits, continue to implement the above changing method.

Request: Implement function

```
expand(string s);
```

Where s is a string with the above form; return the result is a string containing only lowercase letters.

Example:

- String "2(ab3(cde)x)" changes into "abcdecdecdexabcdecdecdex".
- String "2(x0👉)3(z)" changes into "xxzzz".

Note: In this exercise, libraries `iostream`, `string` and `using namespace std`; have been used. You can add other functions for your answer, but you are not allowed to add other libraries.

For example:

Test	Result
<code>cout << expand("2(ab3(cde)x)") << "\n";</code>	abcdecdecdexabcdecdecdex
<code>cout << expand("2(x0(y))3(z)") << "\n";</code>	xxzzz

Answer: (penalty regime: 0 %)

Reset answer

```

1  string repeatString(const string &str, int count) {
2      if (count <= 0) {
3          return "";
4      }
5      return str + repeatString(str, count - 1);
6  }
7  string expandHelper(const string &s, size_t &index) {
8      string result;
9      if (index >= s.length()) {
10         return result;
11     }
12
13     if (isdigit(s[index])) {
14         int repeat = s[index] - '0';
15         index++; // move past the digit
16         if (s[index] == '(') {
17             index++; // move past the '('
18             string subResult = expandHelper(s, index);
19             result += repeatString(subResult, repeat);
20         }
    }

```

```
21  } else if (s[index] == ')') {
22      index++; // move past the ')'
23      return result;
24  } else {
25      result += s[index];
26      index++;
27  }
28
29  return result + expandHelper(s, index);
30 }
31
32 string expand(string s) {
33     size_t index = 0;
34     return expandHelper(s, index);
35 }
```

	Test	Expected	Got	
✓	cout << expand("2(ab3(cde)x)") << "\n";	abcdecdecdecxabcdecdecdecx	abcdecdecdecxabcdecdecdecx	✓
✓	cout << expand("2(x0(y))3(z)") << "\n";	xxzzz	xxzzz	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 7

Đúng

Đạt điểm 1,00 trên 1,00

Give a positive integer x , implement recursive function

```
void printHailstone(int number){}
```

to print the Hailstone Sequence of a given number upto 1 (no space at the end).

Hailstone Sequences follow these rules:

- If a number is even, divide it by 2
- If a number is odd, multiply it by 3 and add 1.

Example:

If number = 5. 5 is odd number so next number is $5*3 + 1 = 16$. 16 is even number so next number is $16/2 = 8 \dots$

Finally, we get Hailstone sequence: 5 16 8 4 2 1.

You can find more information at: <https://diendantoanhoc.net/topic/89145-d%C3%A3y-s%E1%BB%91-hailstone/>

Note: Please note that you can't using key work for, while, goto (even in variable names, comment).

You can implement other recursive functions if needed.

For this exercise, we have `#include <iostream>` and using namespace std;

For example:

Test	Result
printHailstone(32);	32 16 8 4 2 1

Answer: (penalty regime: 0 %)

Reset answer

```

1 void printHailstone(int number)
2 {
3     if (number==1) {cout<< "1"; return;}
4
5     if( number%2==0)
6     { cout<< number<<" ";
7       printHailstone(number/2);}
8
9     else {
10        cout<<number << " ";
11        number*=3;
12        number +=1;
13        cout<< number<<" ";
14        printHailstone(number/2);
15    }
16 }
```

	Test	Expected	Got	
✓	printHailstone(32);	32 16 8 4 2 1	32 16 8 4 2 1	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 8

Đúng

Đạt điểm 1,00 trên 1,00

Function

```
int myArrayToInt(char* str, int n){}
```

takes a **string str** (which represents an positive decimal number), **n** is the number of elements in the string as arguments and returns its value.

Please note that you can't using key work for, while, goto (even in variable names, comment)

For this exercise, we have #include <iostream>, #include <string.h> and using namespace std;

For example:

Test	Result
char str[] = "2020"; printf("%d", myArrayToInt(str, 4));	2020

Answer: (penalty regime: 0 %)

Reset answer

```
1 int myArrayToIntUtil(char* str, int n) {
2     // Trường hợp cơ sở: nếu chỉ còn 1 ký tự, trả về giá trị số của ký tự đó
3     if (n == 1)
4         return str[0] - '0';
5
6     // Đệ quy: chuyển đổi ký tự đầu tiên thành số và cộng với kết quả của phần còn lại
7     return (str[0] - '0') * pow(10, n - 1) + myArrayToIntUtil(str + 1, n - 1);
8 }
9
10 int myArrayToInt(char* str, int n) {
11     return myArrayToIntUtil(str, n);
12 }
```



	Test	Expected	Got	
✓	<pre>char str[] = "2020"; printf("%d", myArrayToInt(str, 4));</pre>	2020	2020	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 9

Đúng

Đạt điểm 1,00 trên 1,00

Give two positive integers a and b, implement function

```
int findLCM(int a, int b){}
```

to find **LCM** (Lowest Common Multiple) of a and b using recursion.

Please note that you can't using key work for, while, goto (even in variable names, comment).

For this exercise, we have #include <iostream> and using namespace std;

For example:

Test	Result
cout << findLCM(10, 102);	510

Answer: (penalty regime: 0 %)

Reset answer

```

1  int findGCD(int a, int b) {
2      if (b == 0)
3          return a;
4      return findGCD(b, a % b);
5  }
6
7  // Hàm tìm LCM sử dụng đệ quy
8  int findLCMUtil(int a, int b, int multiple) {
9      if (multiple % a == 0 && multiple % b == 0)
10         return multiple;
11     return findLCMUtil(a, b, multiple + max(a, b));
12 }
13
14 int findLCM(int a, int b) {
15     return findLCMUtil(a, b, max(a, b));
16 }
17

```



	Test	Expected	Got	
✓	cout << findLCM(10, 102);	510	510	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 10

Đúng

Đạt điểm 1,00 trên 1,00

Given a string `s` consisting only of '(' and ')'.
Your task is to implement a function with following prototype:

```
int mininumBracketAdd(string s);
```

The function returns the minimum number of brackets needed to be inserted to `s` so that the brackets are balanced.

More info:

A sequence of brackets is balanced when there are no unmatched brackets.

Example: `()(())` is balanced, but `))()` is not.

Note:

- The `iostream` library has been used and `namespace std` is being used. No other libraries are allowed.
- Using loop keywords (`for`, `while`, `do`) are not allowed, even in comments and variable names.
- You can write helper functions.

For example:

Test	Result
<code>cout << mininumBracketAdd(")))((");</code>	5

Answer: (penalty regime: 0 %)

[Reset answer](#)

```

1  int mininumBracketAddSupport(const string &str, size_t opencount, size_t closecount, size_t index){
2      if (index == str.size()){
3          return opencount + closecount;
4      }
5
6      char currentChar = str[index];
7
8      if (currentChar == '('){
9          return mininumBracketAddSupport(str, opencount + 1, closecount, index + 1);
10     }
11     else {
12         if (opencount >= 1){
13             return mininumBracketAddSupport(str, opencount - 1, closecount, index + 1);
14         }
15         else {
16             return mininumBracketAddSupport(str, opencount, closecount + 1, index + 1);
17         }
18     }
19 }
20
21 int mininumBracketAdd(const string& str){
22     return mininumBracketAddSupport(str, 0, 0, 0);
23 }
```



	Test	Expected	Got	
✓	<code>cout << mininumBracketAdd(")))((");</code>	5	5	✓
✓	<code>cout << mininumBracketAdd(")))(())()(");</code>	4	4	✓



Passed all tests! ✓

Marks for this submission: 1,00/1,00.



Câu hỏi 11

Đúng

Đạt điểm 1,00 trên 1,00

Given a string `s` representing a sentence consisting only of `a-z` and `A-Z` and space character. Your task is to implement a function with following prototype:

```
string reverseSentence(string s);
```

The function returns the reverse sentence of sentence `s`.

The testcases ensure that there is only one space character between two adjacent words, and the sentences do not begin or end with any space characters.

Note:

- The `iostream` library has been used and `namespace std` is being used. No other libraries are allowed.
- Using loop keywords (`for`, `while`, `do`) are not allowed, even in comments and variable names.
- You can write helper functions.

For example:

Test	Result
<code>cout << reverseSentence("data structure and algorithm is scary");</code>	<code>scary is algorithm and structure data</code>

Answer: (penalty regime: 0, 0, 0, 5, 10, 15, ... %)

Reset answer

```

1 // Hàm trợ giúp để tìm từ tiếp theo trong câu sử dụng đệ quy
2 string getNextWord(const string &s, size_t &index) {
3     if (index >= s.length() || s[index] == ' ') {
4         return "";
5     }
6     string word(1, s[index]);
7     index++;
8     return word + getNextWord(s, index);
9 }
10
11 // Hàm trợ giúp để bỏ qua khoảng trắng
12 void skipSpace(const string &s, size_t &index) {
13     if (index < s.length() && s[index] == ' ') {
14         index++;
15     }
16 }
17
18 // Hàm đệ quy để đảo ngược câu
19 string reverseSentenceHelper(const string &s, size_t &index) {
20     if (index >= s.length()) {
21         return "";
22     }
23     string word = getNextWord(s, index);
24     skipSpace(s, index);
25     string remainingSentence = reverseSentenceHelper(s, index);
26     if (remainingSentence.empty()) {
27         return word;
28     }
29     return remainingSentence + " " + word;
30 }
31
32 string reverseSentence(string s) {
33     size_t index = 0;
34     return reverseSentenceHelper(s, index);
35 }
```

	Test	Expected	Got	
✓	cout << reverseSentence("data structure and algorithm is scary");	scary is algorithm and structure data	scary is algorithm and structure data	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

//



Câu hỏi 12

Đúng

Đạt điểm 1,00 trên 1,00

Given a string, implement function

```
int strLen(char* str){}
```

to calculate length of the string using recursion.

Please note that you can't using key work for, while, goto (even in variable names, comment).

For this exercise, we have #include <iostream> and using namespace std;

For example:

Test	Result
char str[] = "Truong DH Bach Khoa"; cout << strLen(str);	19

Answer: (penalty regime: 0 %)

Reset answer

```
1 int strLen(char* str) {  
2     // Trường hợp cơ sở: nếu ký tự đầu tiên là '\0', độ dài là 0  
3     if (str[0] == '\0') {  
4         return 0;  
5     }  
6     // Trường hợp đệ quy: độ dài của chuỗi là 1 + độ dài của phần còn lại của chuỗi  
7     return 1 + strLen(str + 1);  
8 }
```


	Test	Expected	Got	
✓	<pre>char str[] = "Truong DH Bach Khoa"; cout << strlen(str);</pre>	19	19	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

